

CANDIDAT	DATE	DURÉE	NOTE
		10 minutes	/ 20

LA CONSIGNE

Cochez vos réponses, puis comptez vos points : **le corrigé est en dernière page**. Pandas ne plante pas, il vous donne un résultat faux, en silence. Ce test identifie les 5 pièges les plus courants.

5 questions

une seule bonne réponse

4 points par bonne réponse

★ = piège classique

I LES 5 PIÈGES SILENCIEUX DE PANDAS

questions 1 à 5 ● ● ●

Q1 Vous écrivez `df[df['a']>0]['b'] = 0`. Que se passe-t-il ? ★

- ☐ La colonne b est modifiée en place pour toutes les lignes où `a > 0`.
- ☐ Pandas lève une `KeyError` car la syntaxe est invalide.
- ☐ Chained indexing : `SettingWithCopyWarning`, la modification peut n'avoir aucun effet (écrit sur une copie).
- ☐ Le `DataFrame` original est copié automatiquement avant la modification.

Q2 Vous testez si une valeur est manquante avec `valeur == float('nan')`. Quel est le résultat ? ★

- ☐ True si la valeur est bien NaN.
- ☐ Toujours False : `NaN != NaN` par définition IEEE 754 ; utiliser `.isna()`.
- ☐ Une `TypeError` car `==` ne fonctionne pas sur les flottants.
- ☐ None, ce qui est évalué comme False dans un contexte booléen.

Q3 Vous exécutez `df.merge(autre, on='id')` sans spécifier `how`. Que se passe-t-il pour les lignes sans correspondance ? ★

- ☐ Elles sont conservées avec NaN dans les colonnes manquantes (outer join).
- ☐ Pandas lève une erreur pour signaler les lignes non jointes.
- ☐ Elles disparaissent silencieusement : le comportement par défaut est `how='inner'`.
- ☐ Elles sont ajoutées à la fin du résultat avec un flag `_unmatched`.

Q4 Une colonne contient 10 valeurs dont 3 sont NaN. Que renvoie `col.mean()` ?

- ☐ La moyenne des 10 valeurs (les NaN comptent comme zéro).
- ☐ NaN, car la présence d'un NaN invalide le calcul.
- ☐ La moyenne des 7 valeurs non nulles : `skipna=True` par défaut, piège si l'on attendait une division par 10.
- ☐ Une `ValueError` signalant les valeurs manquantes.

Q5 Vous appliquez `groupby('cat').sum()` sur une colonne dont le dtype est `object` (chaîne de caractères). Que se passe-t-il ? ★

- ☐ Pandas lève une `TypeError` car `sum` est interdit sur du texte.
- ☐ La colonne est ignorée sans avertissement (comportement silencieux depuis Pandas 2.0).
- ☐ Pandas concatène les chaînes ou ignore la colonne : agrégation silencieusement fautive ; vérifier les dtypes avant d'agréger.
- ☐ Pandas convertit automatiquement la colonne en numérique avant d'agréger.

- ✓ **Q1 : Chained indexing · SettingWithCopyWarning, modification sans effet.** `df[df['a']>0]` renvoie une copie ; écrire dessus ne modifie pas le DataFrame d'origine.
Correction : `df.loc[df['a']>0, 'b'] = 0`.
- ✓ **Q2 : Toujours False · NaN != NaN (IEEE 754).** La comparaison `x == float('nan')` renvoie toujours False, même si x est NaN. Utiliser `pd.isna(x)` ou `col.isna()` sur une série.
- ✓ **Q3 : Inner join par défaut · les lignes sans correspondance disparaissent.** Sans `how`, `.merge()` fait un `inner join` : toute ligne absente de l'une ou l'autre table est supprimée silencieusement. Vérifier `len()` avant et après la fusion.
- ✓ **Q4 : Moyenne sur 7 valeurs · skipna=True par défaut.** `.mean()` ignore les NaN (divise par 7, pas par 10). Piège fréquent en calcul de taux ou de moyenne pondérée : la population réelle n'est pas celle qu'on croit.

- ✓ **Q5 : Agrégation silencieusement fausse · vérifier les dtypes.** Selon la version Pandas et le contenu, `sum()` peut concaténer des chaînes ou sauter la colonne sans avertissement. Toujours appliquer `df.dtypes` ou `df.select_dtypes(include='number')` avant un `groupby`.

VOTRE NOTE SUR 20

0-8

Les pièges silencieux vous coûtent du temps en production : commencez par l'indexation (`.loc`) et les `dtypes`.

12

Bon niveau opérationnel. Renforcez la gestion des NaN et les comportements par défaut des agrégations.

16-20

Vous savez où Pandas ment. Votre code en production est fiable.

→ **Postez votre note en commentaire, sans tricher.**